AUMS SYNOPSIS REPORT

# ProveOnce

## Privacy-Preserving Eligibility Verification Using Blockchain and Zero-Knowledge Proofs

| Field | Value |
|---|---|
| Project type | Web3 / Zero-Knowledge Proof application |
| Repository | Sasi-Kiran-Yasaswi-Vejju/ProooveOnce |
| Date | 15 October 2026 |
| Core result | YES or NO without raw certificate data |

# 1. Abstract and Problem

ProveOnce is a privacy-preserving eligibility verification system for academic and government credentials. It lets a student prove that a private credential satisfies a condition such as “CGPA is at least 7.5” or “family income is at most ■8,00,000” without revealing the complete certificate or exact value.

The issuer signs the credential and only a cryptographic commitment is stored on-chain. The student generates a Groth16 proof when a verifier asks a question. The Solidity contract verifies it and records only YES or NO.

# 2. Objectives

• Create signed credentials for trusted issuers.
• Keep raw certificate fields off-chain.
• Support exact verifier threshold queries.
• Reject forged, replayed, copied, expired, or revoked proofs.
• Support anonymous Merkle membership and nullifiers.

# 3. Users and Use Cases

| User | Responsibility |
|---|---|
| Issuer | Signs credentials and registers commitments. |
| Student | Keeps private data and generates proofs. |
| Verifier | Asks a question and receives YES or NO. |

Use cases include student loans, scholarships, employment screening, admissions, government benefits, and income/category eligibility.

# 4. Main Idea

The credential is like a sealed envelope. The blockchain stores its fingerprint, not its contents. The student produces a mathematical receipt proving that the hidden contents satisfy the verifier’s question; the receipt reveals only the answer.

# 5. Architecture and Workflow

| 1 | → | 2 | → | 3 | → | 4 | → | 5 |
|---|---|---|---|---|---|---|---|---|
| Issuer signs | → | Poseidon commitment | → | Blockchain registry | → | ZK proof | → | Solidity YES/NO |

# 6. End-to-End Implementation

**Credential issuance:** A BabyJubJub issuer key signs a Poseidon commitment calculated from credential fields and a random salt. The student keeps fields, salt, and signature private; only the commitment is registered.

**Query and proof:** The verifier creates a query such as CGPA >= 750. The Circom circuit proves the commitment, signature, hidden comparison, query ID, and holder binding without exposing the witness.

**On-chain verification:** EligibilityVerifier.sol checks proof validity, issuer, credential status, expiry/revocation, query binding, and holder, then records only true or false.

**Anonymous Phase 4:** Merkle roots hide the student's leaf position and a per-query nullifier prevents double submission.

# 7. Cryptographic Components

| Component | Purpose |
|---|---|
| Poseidon | ZK-friendly commitment hash. |
| Salt | Prevents guessing values from a commitment. |
| BabyJubJub + EdDSA | Issuer signature scheme for the circuit. |
| Circom + snarkjs | Circuit, setup, proof generation, and verifier export. |
| Groth16 | Compact proof system checked by Solidity. |

# 8. Tools, Modules, and Integration

| Tool / module | Implementation |
|---|---|
| React + Vite | Student Vault, Issuer Hub, Verifier Portal, proof status, themes. |
| Solidity + Hardhat | Contracts, local chain, tests, compilation, deployment. |
| Node.js + Express | Local API bridge in scripts/server.js. |
| Circom + snarkjs | Real ZK circuit and Groth16 proof path. |
| Hardhat signers | Local issuer, student, bank, and owner roles. |
| Sepolia | Deployment supported by scripts/deploy.js. |

# 9. Smart Contracts

| Contract | Purpose |
|---|---|
| IssuerRegistry.sol | Active issuers and public signing keys. |
| CredentialRegistry.sol | Commitments, expiry, and revocation. |
| EligibilityVerifier.sol | Normal proof verification and YES/NO. |
| MerkleCredentialRegistry.sol | Incremental Merkle tree and roots. |
| EligibilityVerifierV2.sol | Anonymous verification and nullifiers. |
| DKIMRegistry.sol | Email-domain key metadata. |
| Groth16Verifier.sol | Generated pairing verifier. |

# 10. Testnet and Wallet Accuracy Note

Sepolia deployment support and addresses are recorded in deployed-addresses.json. The repeatable presentation path uses the local Hardhat blockchain, where signers represent issuer, student, bank, and owner. The React wallet button is currently a presentation UI state and does not yet perform a complete MetaMask transaction flow.

This should be stated honestly: the project demonstrates UI, contracts, local wallet roles, ZK proofs, tests, and Sepolia deployment support; direct MetaMask-to-Sepolia frontend integration remains future work.

# 11. Execution and Verified Results

From the repository root, generate ZK artifacts once with Circom, then run:

```
npm run server # terminal 1
npm run ui # terminal 2
open http://localhost:5173
npm test
npm run demo
```

The real demo creates credentials, asks four questions, generates proofs, and verifies them on-chain.

| Validation | Observed result |
|---|---|
| Solidity compilation | Passed |
| Hardhat contract tests | 14 passing |
| React production build | Passed |
| Local API status | Passed |
| Anonymous Merkle demo | Passed |
| Real Groth16 demo | Passed |

## Expected output

```
Q1 CGPA >= 7.50 -> YES
Q2 CGPA >= 9.00 -> NO
Q3 Family income <= ■8,00,000 -> YES
Q4 CGPA >= 7.50 AND no backlogs -> YES
```

# 12. Rubric Mapping

| Rubric | Evidence |
|---|---|
| Output and Execution — 5 | Show UI, proof output, npm run demo, and 14 passing tests. |
| Tools Integration — 5 | Explain UI, Hardhat signers, Solidity, Circom/Groth16, API, Sepolia. |
| Oral Presentation — 3 | Explain issuer → commitment → proof → contract → YES/NO. |
| Report — 2 | Submit this synopsis. |

# 13. Limitations and Conclusion

The local chain is temporary, the trusted setup is for development, issuers remain trusted, and direct MetaMask/Sepolia frontend transactions are future work. ProveOnce demonstrates that blockchain and zero-knowledge proofs can verify eligibility without exposing unnecessary personal data.